# PokerLab Phase 1 Code Reference

*A future-reference guide to the current Java/Maven poker engine boilerplate*

**Purpose:** This document explains the current Phase 1 codebase: what each class is responsible for, how the 5-card and 7-card hand evaluation works, how the CLI demo flows through the system, and what should be improved next.

**Current milestone:** Phase 1 - Core Poker Engine. The project understands cards, decks, poker hand categories, hand comparison, and simple heads-up Texas Hold'em showdown evaluation.

## Contents


- 1. High-level overview
- 2. Project structure
- 3. Build and run configuration
- 4. Core card package
- 5. Hand evaluation package
- 6. Showdown package
- 7. CLI application flow
- 8. Tests currently included
- 9. End-to-end example
- 10. Current limitations and improvement checklist
- 11. Phase 2 preparation notes


## 1. High-level overview

**What this codebase currently does:** it provides a clean, testable foundation for a poker analysis engine. It does not yet run equity simulations or solve strategy. Instead, it builds the basic engine needed before Monte Carlo simulation, range analysis, EV tools, or CFR can work.

- Represents ranks, suits, and individual cards.
- Builds and manipulates a standard 52-card deck.
- Evaluates exactly 5-card poker hands.
- Evaluates the best 5-card hand from 5, 6, or 7 cards.
- Compares a hero hand and villain hand on a shared board.
- Provides a small command-line demo and starter unit tests.

**Design principle:** The most important design idea is separation of responsibilities. The card package knows only about cards and decks; the hand package knows how to rank poker hands; the showdown package combines private cards and board cards to decide a winner. This makes the code easier to test and extend.

## 2. Project structure

```
pokerlab-phase1/
├── pom.xml
├── README.md
├── src/main/java/com/pokerlab/
│   ├── app/
│   │   └── Main.java
│   └── core/
│       ├── card/
│       │   ├── Card.java
│       │   ├── Deck.java
│       │   ├── Rank.java
│       │   └── Suit.java
│       ├── hand/
│       │   ├── EvaluatedHand.java
│       │   ├── HandCategory.java
│       │   ├── HandEvaluator.java
│       │   └── HandRank.java
│       └── showdown/
│           ├── Showdown.java
│           ├── ShowdownResult.java
│           └── Winner.java
└── src/test/java/com/pokerlab/
    └── starter JUnit tests
```

| Package | Purpose | Main files |
|---|---|---|
| com.pokerlab.app | Small command-line entry point used to manually run a sample showdown. | Main.java |
| com.pokerlab.core.card | Basic poker primitives: cards, ranks, suits, and a deck. | Card, Rank, Suit, Deck |
| com.pokerlab.core.hand | Poker hand categories, ranking, comparison, and evaluation logic. | HandCategory, HandRank, EvaluatedHand, HandEvaluator |
| com.pokerlab.core.showdown | Heads-up Hold'em comparison between hero and villain on a shared board. | Showdown, ShowdownResult, Winner |
| src/test | Unit tests that protect basic behaviour from breaking as the project grows. | CardTest, DeckTest, HandEvaluatorTest, ShowdownTest |

## 3. Build and run configuration

### 3.1 pom.xml

**pom.xml** is the Maven project configuration file. Maven uses it to know the project name, Java version, dependencies, and plugins.

| Setting | Current value | Meaning |
|---|---|---|
| groupId | com.pokerlab | The organisation/package-style identifier for the project. |
| artifactId | pokerlab-phase1 | The build artifact name. |
| version | 1.0.0-SNAPSHOT | A development version; SNAPSHOT means it is still changing. |
| Java version | 17 | The code is compiled using Java 17 language features. |
| JUnit Jupiter | 5.10.2 | Test framework used for unit tests. |

| Setting | Current value | Meaning |
|---|---|---|
| maven-surefire-plugin | 3.2.5 | Runs JUnit tests when you execute mvn test. |
| exec-maven-plugin | 3.1.0 | Runs the CLI app with mvn exec:java. |

```
# Run all unit tests
mvn test

# Run the sample command-line demo
mvn exec:java

# Run with custom hero, villain, and board cards
mvn exec:java -Dexec.args="AsKs QdQc Ah7c2s9dJc"
```

**Card notation:** The current CLI expects compact two-character notation: As = Ace of spades, Qd = Queen of diamonds, Tc = Ten of clubs. Ten is written as T rather than 10.

## 4. Core card package

This package contains the lowest-level building blocks. These classes should stay simple because everything else depends on them.

| Class | Type | Responsibility |
|---|---|---|
| Rank | enum | Stores the 13 card ranks from Two to Ace, including both display symbol and numeric value. |
| Suit | enum | Stores the 4 suits and their compact/display symbols. |
| Card | record | Immutable pair of rank and suit. Can parse compact two-character notation such as A-suited-spades style input (As) or Qd. |
| Deck | class | Mutable standard 52-card deck with shuffle, deal, remove, and copy methods. |

### 4.1 Rank.java

**Rank** is an enum. An enum is useful here because there are only 13 possible ranks. Each rank stores two pieces of data:

- symbol: the compact character used for input/output, e.g. A, K, Q, J, T, 9.
- value: the numeric strength used by the evaluator, where Two = 2 and Ace = 14.

```
ACE  -> symbol A, value 14
TEN  -> symbol T, value 10
TWO  -> symbol 2, value 2
```

**fromChar(char value)** turns user input into a Rank. It uppercases the input first, so both a and A would work for ace.

## 4.2 Suit.java

**Suit** is another enum. It stores:

- symbol: compact input form, e.g. s, h, d, c.
- displaySymbol: nicer output form, e.g. ♠, ♥, ♦, ♣.

```
SPADES   -> compact s, display ♠
HEARTS   -> compact h, display ♥
DIAMONDS -> compact d, display ♦
CLUBS    -> compact c, display ♣
```

**fromChar(char value)** lowercases the input, so S and s both parse as spades.

## 4.3 Card.java

**Card** is a Java record:

```
public record Card(Rank rank, Suit suit)
```

**Why a record works well here:** a card is just immutable data. Once a Card is created, its rank and suit should not change. Records also automatically provide equals, hashCode, and accessors such as rank() and suit().

| Method | Purpose |
|---|---|
| Card.of(Rank rank, Suit suit) | Convenience factory method for creating a card. |
| Card.parse(String text) | Parses compact notation such as As, Qd, Tc. |
| compact() | Returns compact notation such as As. |
| toString() | Returns display notation such as A♠. |

```
Card card = Card.parse("As");
card.rank();     // ACE
card.suit();     // SPADES
card.compact();  // "As"
card.toString(); // "A♠"
```

**Input rule:** Card.parse intentionally rejects 10s because the project uses T for ten. This keeps every card exactly two characters long, which makes parsing a string like AsKs easy.

## 4.4 Deck.java

**Deck** creates a standard 52-card deck in its constructor by looping through every suit and every rank.

```
for (Suit suit : Suit.values()) {
    for (Rank rank : Rank.values()) {
        cards.add(Card.of(rank, suit));
    }
}
```

| Method | What it does | Important detail |
|---|---|---|
| size() | Returns number of cards left. | Useful for tests and simulation loops. |
| isEmpty() | Checks whether the deck has no cards left. | Used before dealing or in future simulations. |
| shuffle() | Randomly shuffles the deck. | Uses Java's default randomness. |

| Method | What it does | Important detail |
|---|---|---|
| shuffle(Random random) | Shuffles with a supplied Random object. | Useful for repeatable tests and reproducible simulations. |
| deal() | Removes and returns one card. | Deals from the end of the internal list. |
| deal(int count) | Deals multiple cards. | Validates that count is non-negative and not larger than the deck. |
| remove(Card card) | Removes a known card. | Will be useful in Phase 2 when hero/villain/board cards must be removed before simulation. |
| cards() | Returns a defensive copy. | Outside code cannot directly mutate the deck list. |

**Mutability:** Deck is intentionally mutable. Dealing a card changes the deck. Card is immutable, but Deck represents changing state.

# 5. Hand evaluation package

This is the most important package in Phase 1. It decides what a poker hand is and how two poker hands compare.

| Class | Type | Responsibility |
|---|---|---|
| HandCategory | enum | Names every poker hand category and assigns a strength value. |
| HandRank | record | Stores category plus tiebreaker values; implements comparison. |
| EvaluatedHand | record | Stores the HandRank and the actual 5 cards used. |
| HandEvaluator | utility class | Contains evaluateFive and evaluateBest logic. |

## 5.1 HandCategory.java

**HandCategory** lists the poker hand types from weakest to strongest. Each category has a numeric strength.

| Category | Strength | Meaning |
|---|---|---|
| HIGH_CARD | 0 | No pair, no straight, no flush. |
| ONE_PAIR | 1 | Two cards of the same rank. |
| TWO_PAIR | 2 | Two separate pairs. |
| THREE_OF_A_KIND | 3 | Three cards of the same rank. |
| STRAIGHT | 4 | Five consecutive ranks. |
| FLUSH | 5 | Five cards of the same suit. |
| FULL_HOUSE | 6 | Three of a kind plus a pair. |
| FOUR_OF_A_KIND | 7 | Four cards of the same rank. |
| STRAIGHT_FLUSH | 8 | Straight and flush together. |
| ROYAL_FLUSH | 9 | Ace-high straight flush. |

## 5.2 HandRank.java

**HandRank** is the comparison object. A poker hand is not ranked only by category; it also needs tiebreakers.

```
public record HandRank(HandCategory category, List<Integer> tiebreakers)
        implements Comparable<HandRank>
```

| Example hand | Category | Tiebreakers | Why |
|---|---|---|---|
| A♠ A♥ K♦ 7♣ 2♠ | ONE_PAIR | [14, 13, 7, 2] | Pair of aces, then kickers K, 7, 2. |
| K♠ K♥ A♦ 7♣ 2♠ | ONE_PAIR | [13, 14, 7, 2] | Pair of kings loses to pair of aces. |
| A♠ K♠ Q♠ J♠ T♠ | ROYAL_FLUSH | [14] | Ace-high straight flush. |
| 5♠ 4♦ 3♣ 2♥ A♠ | STRAIGHT | [5] | Wheel straight; ace counts low, so high card is 5. |

**Comparison process:**

1. Compare category strength first. A flush always beats a straight, regardless of high card.
2. If categories match, compare tiebreakers from left to right.
3. The first different tiebreaker decides the winner.
4. If everything matches, the hands tie.

## 5.3 EvaluatedHand.java

**EvaluatedHand** wraps the result of evaluation. It stores both the rank information and the exact cards used to make the best hand.

| Field/method | Purpose |
|---|---|
| HandRank rank | The sortable category + tiebreakers. |
| List<Card> cards | The five cards used for that hand. |
| category() | Shortcut for rank.category(). |
| compareTo(EvaluatedHand other) | Delegates comparison to HandRank. |
| toString() | Readable output such as One pair using [A♠, K♠, A❤️, 7♣, 2♠]. |

**Immutability:** The cards list is defensively copied using List.copyOf(cards), so other code cannot mutate the evaluated hand after it is created.

## 5.4 HandEvaluator.java

**HandEvaluator** is a final utility class with a private constructor. That means you do not create HandEvaluator objects. You simply call static methods such as HandEvaluator.evaluateFive(cards).

| Public method | Input | Output | Use case |
|---|---|---|---|
| evaluateFive | Exactly 5 cards | EvaluatedHand | Classify a normal 5-card poker hand. |
| evaluateBest | 5, 6, or 7 cards | Best EvaluatedHand | Texas Hold'em: choose best 5-card hand from hole cards + board. |

### 5.4.1 evaluateFive: how a 5-card hand is classified

**evaluateFive** starts by validating the input:

- The list must not be null.
- There must be exactly 5 cards.
- There must be no duplicate cards.

**Then it calculates three useful pieces of information:**

| Variable/concept | How it is calculated | Why it matters |
|---|---|---|
| flush | All suits are the same. | Needed for flush, straight flush, and royal flush. |
| straightHigh | Checks whether ranks form five consecutive values. | Needed for straight and straight flush. |

| Variable/concept | How it is calculated | Why it matters |
|---|---|---|
| countsByRank | Counts how many of each rank are present. | Needed for pairs, trips, full house, and quads. |
| groups | Ranks sorted by count first, then rank value. | Makes it easy to identify the most important repeated rank. |
| ranksDescending | All rank values sorted high to low. | Used for flush and high-card tiebreakers. |

**The evaluator then checks categories from strongest patterns downwards:**

```
if straight and flush:
    royal flush or straight flush
else if four cards share a rank:
    four of a kind
else if three cards share a rank and two cards share another rank:
    full house
else if flush:
    flush
else if straight:
    straight
else if three cards share a rank:
    three of a kind
else if two pair ranks exist:
    two pair
else if one pair rank exists:
    one pair
else:
    high card
```

**Why order matters:** The exact Java code uses if statements rather than this pseudo-code, but this is the logical order. Checking stronger/special cases first avoids misclassifying a full house as only three of a kind, or a straight flush as only a straight.

### 5.4.2 Tiebreakers used by each category

| Category | Tiebreaker list |
|---|---|
| ROYAL_FLUSH | [14] |
| STRAIGHT_FLUSH | [straight high card] |
| FOUR_OF_A_KIND | [quad rank, kicker] |
| FULL_HOUSE | [trip rank, pair rank] |
| FLUSH | [all five ranks descending] |
| STRAIGHT | [straight high card] |
| THREE_OF_A_KIND | [trip rank, highest kicker, second kicker] |
| TWO_PAIR | [higher pair, lower pair, kicker] |
| ONE_PAIR | [pair rank, highest kicker, second kicker, third kicker] |
| HIGH_CARD | [all five ranks descending] |

### 5.4.3 Straight detection and the wheel straight

**straightHighCard** returns null if the cards are not a straight. If they are a straight, it returns the high card value.

```
A K Q J T -> straightHigh = 14
9 8 7 6 5 -> straightHigh = 9
A 5 4 3 2 -> straightHigh = 5
```

**Important poker rule:** The A-2-3-4-5 straight is called the wheel. Although Ace normally has value 14, it is treated as low in this one special case, so the straight high card is 5.

### 5.4.4 evaluateBest: how 7-card Hold'em evaluation works

**evaluateBest** accepts 5, 6, or 7 cards. For Texas Hold'em, a player can have 2 hole cards plus up to 5 board cards, so there are usually 7 cards available at showdown.

**Simple Phase 1 strategy:** generate every possible 5-card combination, evaluate each one using evaluateFive, and keep the best result.

```
7 cards choose 5 = 21 possible 5-card hands

For each 5-card candidate:
    evaluated = evaluateFive(candidate)
    if best is null or evaluated beats current best:
        best = evaluated

return best
```

**Why this is good for Phase 1:** it is easy to understand, hard to get wrong, and fast enough for early development. Later, if Monte Carlo simulations need more speed, this can be replaced or supplemented with bit masks, lookup tables, or cached results.

### 5.4.5 Validation helpers

| Helper | Purpose |
|---|---|
| validateCardCount(cards, min, max) | Rejects null lists and wrong card counts. |
| validateNoDuplicates(cards) | Rejects duplicate cards because the same physical card cannot appear twice. |
| isFlush(cards) | Checks whether all cards have the same suit. |
| join(first, rest) | Builds a tiebreaker list such as [pair rank, kicker1, kicker2, kicker3]. |

# 6. Showdown package

The showdown package connects the evaluator to a poker situation: hero has 2 private cards, villain has 2 private cards, and both share the same board.

| Class | Type | Responsibility |
|---|---|---|
| Winner | enum | Represents HERO, VILLAIN, or TIE. |
| ShowdownResult | record | Stores hero evaluated hand, villain evaluated hand, and winner. |
| Showdown | utility class | Validates a heads-up board and compares both players. |

## 6.1 Showdown.java

**Showdown.compare(heroHoleCards, villainHoleCards, boardCards)** is the main method.

1. Validate hero has exactly 2 hole cards.
2. Validate villain has exactly 2 hole cards.
3. Validate board has between 3 and 5 cards.
4. Combine all cards and check for duplicates.
5. Build heroCards = hero hole cards + board.
6. Build villainCards = villain hole cards + board.
7. Evaluate each player's best hand using HandEvaluator.evaluateBest.
8. Compare the two evaluated hands.
9. Return ShowdownResult with the winner.

```
EvaluatedHand heroBest = HandEvaluator.evaluateBest(heroCards);
EvaluatedHand villainBest = HandEvaluator.evaluateBest(villainCards);

int comparison = heroBest.compareTo(villainBest);
Winner winner = comparison > 0 ? Winner.HERO
              : comparison < 0 ? Winner.VILLAIN
              : Winner.TIE;
```

**Board size:** Allowing board size between 3 and 5 means the same method can compare hands on the flop, turn, or river. At 3 or 4 board cards, evaluateBest checks the best hand from 5 or 6 total cards.

## 6.2 ShowdownResult.java and Winner.java

**ShowdownResult** is just a clean return object. Instead of returning only HERO or VILLAIN, it also stores both evaluated hands so the UI/CLI can explain why someone won.

```
public record ShowdownResult(
    EvaluatedHand heroHand,
    EvaluatedHand villainHand,
    Winner winner
)
```

**Winner** is intentionally simple:

```
public enum Winner {
    HERO,
    VILLAIN,
    TIE
}
```

## 7. CLI application flow

The CLI app is not the final product. It is a simple way to prove the engine works before building a UI or Phase 2 simulator.

| Part | Explanation |
|---|---|
| main(String[] args) | Entry point when the program runs. |
| args.length == 3 | User supplied hero, villain, and board strings. |
| else branch | Runs a default sample showdown if no arguments are provided. |
| parseCards | Converts a compact string such as AsKs into a List<Card>. |
| Showdown.compare | Runs the actual poker comparison. |
| System.out.println | Prints the inputs, both best hands, and the winner. |

```
# Default sample
mvn exec:java

# Custom example
mvn exec:java -Dexec.args="AsKs QdQc Ah7c2s9dJc"
```

**The compact input strings are split every two characters.** For example, AsKs becomes As and Ks. Each two-character chunk is passed into Card.parse.

```
for (int i = 0; i < cleaned.length(); i += 2) {
    cards.add(Card.parse(cleaned.substring(i, i + 2)));
}
```

## 8. Tests currently included

The test suite is small, but it already protects the most important early behaviours. These tests should be expanded heavily before moving into Phase 2.

| Test class | What it checks |
|---|---|
| CardTest | Card parsing works for compact notation; invalid notation is rejected. |
| DeckTest | A new deck contains 52 unique cards; dealing cards reduces deck size. |
| HandEvaluatorTest | Recognises royal flush, wheel straight, category comparison, and best 5 from 7. |
| ShowdownTest | Hero pair of aces beats villain pair of queens in the sample Hold'em scenario. |

**Most important test to understand:** the evaluator test that checks best 5 from 7 proves the nested-combination approach works for Hold'em-style evaluation.

```
EvaluatedHand best = HandEvaluator.evaluateBest(
    cards("As", "Ks", "Qs", "Js", "Ts", "2d", "3c")
);

assertEquals(HandCategory.ROYAL_FLUSH, best.category());
assertEquals(5, best.cards().size());
```

## 9. End-to-end example

**Input:** Hero has A♠ K♠, villain has Q♦ Q♣, and the board is A♥ 7♣ 2♠ 9♦ J♣.

```
mvn exec:java -Dexec.args="AsKs QdQc Ah7c2s9dJc"
```

| Step | What happens |
|---|---|
| 1 | Main receives three strings: AsKs, QdQc, Ah7c2s9dJc. |
| 2 | parseCards converts them into List<Card> objects. |
| 3 | Showdown.compare validates card counts and duplicate cards. |
| 4 | Hero cards become A♠ K♠ + board. |
| 5 | Villain cards become Q♦ Q♣ + board. |
| 6 | HandEvaluator.evaluateBest checks each player's best 5-card hand. |
| 7 | Hero's best category is ONE_PAIR: pair of aces. |
| 8 | Villain's best category is ONE_PAIR: pair of queens. |
| 9 | HandRank comparison sees pair rank 14 beats pair rank 12. |
| 10 | Winner is HERO. |

**Why this example matters:** This example is useful as a sanity check because both players have the same hand category. The result depends on tiebreakers, not just category strength.

## 10. Current limitations and improvement checklist

This is a good Phase 1 baseline, but it is still early-stage code. These are the most important next improvements.

| Area | Current limitation | Recommended improvement |
|---|---|---|
| CLI validation | It checks string length, but error messages are still basic. | Add friendlier messages and examples when input fails. |
| Duplicate reporting | HandEvaluator reports counts map for duplicates; Showdown only gives a general message. | Report exactly which card was duplicated. |
| Test coverage | Only a few hand categories are tested. | Add tests for every category and edge-case comparisons. |
| Performance | 7-card evaluation brute-forces all combinations. | Keep for now, then benchmark before optimising. |
| Card ordering | EvaluatedHand stores cards in the candidate order, not necessarily pretty sorted order. | Optionally sort display cards by relevance for nicer output. |
| Range support | No poker range parser yet. | Add in Phase 3 after equity simulation basics. |
| Simulation | No Monte Carlo engine yet. | Phase 2 should add a simulator that removes known cards from the deck and samples unknown boards. |
| UI | Only CLI output exists. | Add UI only after the engine and tests are stronger. |

## 11. Phase 2 preparation notes

**Why Phase 1 matters for Phase 2:** a Monte Carlo equity simulator repeatedly deals random unknown cards and uses the evaluator to decide who wins. Therefore, Card, Deck, HandEvaluator, and Showdown are the core dependencies for the next stage.

| Phase 2 feature | Phase 1 code it will reuse |
|---|---|
| Hand-vs-hand equity | Card.parse, Deck.remove, Deck.shuffle, Deck.deal, Showdown.compare |
| Random board completion | Deck after known cards have been removed. |
| Win/tie/loss counting | Winner enum from ShowdownResult. |
| Equity formula | wins, ties, and simulation count collected around Showdown.compare. |
| Reproducible simulations | Deck.shuffle(Random random) for deterministic test seeds. |

```
future Monte Carlo loop idea:

known cards = hero + villain + partial board
for simulation in 1..N:
    deck = new Deck()
    remove all known cards from deck
    shuffle deck
    complete board to 5 cards
    result = Showdown.compare(hero, villain, completedBoard)
    count hero wins, villain wins, ties
```

**Engineering advice:** Do not optimise the hand evaluator before measuring performance. First, build the simulator with the clear evaluator, write tests, then benchmark. Only optimise if the runtime is actually a problem.

## Appendix A: Quick class responsibility summary

| File | One-sentence description |
|---|---|
| Main.java | Parses CLI input, runs a showdown, and prints the result. |
| Card.java | Immutable rank+suit object with compact parsing and display output. |
| Rank.java | Enum for ranks Two through Ace, including numeric strength. |
| Suit.java | Enum for clubs, diamonds, hearts, spades, including display symbols. |
| Deck.java | Mutable 52-card deck with shuffle, deal, remove, and copy behaviour. |
| HandCategory.java | Ordered list of poker hand categories from high card to royal flush. |
| HandRank.java | Comparable category + tiebreakers object used to decide which hand is stronger. |
| EvaluatedHand.java | The evaluated result: hand rank plus the five cards used. |
| HandEvaluator.java | Core logic for classifying 5-card hands and choosing best 5 from 7. |
| Showdown.java | Combines hole cards with board cards and compares hero vs villain. |
| ShowdownResult.java | Return object containing both evaluated hands and the winner. |
| Winner.java | Enum representing HERO, VILLAIN, or TIE. |

## Appendix B: Suggested immediate next coding tasks

- Add evaluator tests for every category: high card, pair, two pair, trips, straight, flush, full house, quads, straight flush, royal flush.
- Add comparison tests for tricky tiebreakers: pair vs pair, two pair vs two pair, flush vs flush, full house vs full house.
- Add duplicate-card tests for Showdown.compare and HandEvaluator.evaluateBest.
- Add a small EquityResult record in preparation for Phase 2.
- Build a MonteCarloSimulator class that supports fixed hero/villain hands and unknown board completion.
- Keep the CLI simple for now; the core engine is more important than UI polish at this stage.

# Code review confirmation

The project owner reviewed the code changes in the revision below and explicitly cleared PR creation on 2026-11-04 (Europe/London).

Reviewed code commit

`2b9d114ea3999c6224e04daeba6261afb6ca44e8`

Branch: codex/physical-sequence-root-minimum

Clearance message: cleared 2b9d114

This confirmation applies to the code changes in that revision. It does not assert review of the entire historical codebase or later changes. Further material code changes require renewed review.

The project Code Review History records this approval and earlier reviews.